CIRCUIT

Programmable strategies for dreamDEX Event Contracts

Product Requirements Document v1.0 — LOCKED

HACKATHON	Somnia × dreamDEX Event Contracts Hackathon
NETWORK	Somnia Shannon Testnet
DATE	4 September 2026
STATUS	Locked for implementation

One-line product definition

Circuit turns a user's trading intent into a bounded, programmable strategy that can automatically execute across rolling dreamDEX Event Contracts, with Somnia Reactivity driving event-based progression and Somnia Agents providing an optional natural-language strategy compiler.

POSITIONING

Other projects build strategies on dreamDEX. Circuit makes strategies programmable.

Contents

0. Document Lock

This document defines the hackathon version of Circuit. Product scope, product thesis, core architecture, P0 requirements, and demo narrative are locked unless a hard technical blocker is discovered during implementation.

The team must not add unrelated sponsor technologies, extra chains, social features, portfolio management, copy-trading economics, pooled vaults, backtesting infrastructure, or generalized DeFi automation before all P0 acceptance criteria pass.

If time becomes constrained, the cut order is:

1. Share/clone strategy page
2. Extra strategy templates
3. Historical performance charts
4. Additional assets/cadences beyond the primary demo series
5. Advanced visual-builder editing affordances

The following may **not** be cut:

- Real dreamDEX Event Contract execution on Somnia testnet
- A real Somnia **on-chain Reactivity subscription**
- Bounded strategy policy enforcement
- A clear visual strategy representation
- End-to-end strategy state progression
- Emergency pause/revocation path
- One small, real Somnia Agent integration
- A polished 2–3 minute demo flow

1. Executive Summary

Circuit is a programmable strategy layer for dreamDEX Event Contracts.

Today, a dreamDEX Event Contract presents a single market decision: choose Up or Down for a fixed rolling window. Builders can create bots, rollover products, conditional sequences, hedging systems, vaults, and games on top of that primitive, but each product typically hard-codes one strategy or one workflow.

Circuit changes the abstraction.

Instead of building one strategy, Circuit lets a user define a **strategy graph** composed of simple primitives:

- observe a market condition
- buy UP or DOWN
- react to resolution
- roll a percentage into the next window
- stop after a bounded number of rounds, losses, or capital-at-risk

A strategy such as:

> "If UP trades above 70%, buy DOWN with 10 units of collateral. If I win, roll 50% into the next BTC 15m market. Stop after two losses or 20 units of total capital-at-risk."

becomes a deterministic graph that the user reviews before activation.

Somnia Agents may convert natural language into a proposed graph, but **AI never receives discretionary control over funds**. The Agent is a compiler-like UX layer. Execution is bounded by deterministic rules approved by the user.

Somnia Reactivity provides the event-driven primitive. Circuit uses an on-chain subscription so market activity/resolution can trigger strategy transitions without a conventional polling loop. dreamDEX provides the execution venue, market lifecycle, order book, outcome positions, and settlement.

The hackathon MVP is intentionally narrow: one excellent programmable strategy engine, one polished builder, one real on-chain Reactivity path, and one end-to-end multi-window demo.

2. Product Thesis

2.1 Problem

Event Contracts are composable, but strategy logic is fragmented across applications.

A user who wants a multi-step policy such as:

- wait for a probability threshold
- take the opposite side
- roll only profits
- stop after two losses
- cap total exposure

normally needs a bespoke bot or application.

The same is true for builders. Each new product repeatedly implements market discovery, lifecycle tracking, trigger logic, risk limits, execution, settlement handling, and rollover behavior.

2.2 Insight

The reusable product is not another trading bot.

The reusable product is a **small strategy language + deterministic execution engine** for Event Contracts.

2.3 Why dreamDEX

dreamDEX Event Contracts have the properties Circuit needs:

- binary UP/DOWN outcomes

- rolling BTC/ETH windows
- an on-chain CLOB
- programmable trading through the `@somnia-chain/markets-sdk`
- on-chain market status and settlement
- ERC-6909 outcome positions
- automatic successor windows

2.4 Why Somnia Reactivity

Circuit is inherently event-driven. A strategy should progress because an on-chain event happened, not because a centralized server repeatedly asks whether something changed.

For the hackathon MVP, Circuit must use a **real on-chain Reactivity subscription** whose handler updates or advances Circuit strategy state.

2.5 Why Somnia Agents

Agents are useful only where non-deterministic interpretation improves UX.

Circuit uses an Agent narrowly:

natural-language intent → proposed canonical strategy manifest

The output must pass schema validation and must be shown to the user before activation.

The Agent must not:

- autonomously invent risk limits
- change an active strategy
- bypass user approval
- custody funds
- directly decide trades outside approved strategy rules

3. Competitive Positioning

As of 4 September 2026, the active hackathon field already includes AI trading systems, rollover strategies, conditional sequences, risk systems, market-making infrastructure, consumer games, pooled strategy vaults, and trading UX products.

Circuit must not compete by being “another AI bot” or “another auto-roll app.”

Existing direction	Representative active BUIDLs	Circuit differentiation
Fixed multi-window instrument	Runs	Circuit is the language/engine for creating many multi-window behaviors
Fixed conditional sequence	Branch	Conditional paths are one Circuit graph pattern, not the whole product
Fixed rollover policy	Let It Ride	`ROLL_PERCENT` + stop policies are reusable primitives
Policy-controlled hedging	KasuwaShield	Circuit generalizes bounded policies beyond one hedge use case
Shared capital strategy	DreamVault	Circuit is user-owned, strategy-specific, and does not require pooled custody
Market making primitive	HOUSE / rampart	Complementary infrastructure rather than Circuit’s product category
AI analysis/trading	Eventra, DreamDesk, DreamSentinel, Rivo, Vitamin M, QDS	Circuit uses AI as a compiler, not as an opaque trading oracle
Gamified consumer trading	Tock, StreakTrader, Market Dungeon	Circuit is a programmable strategy protocol/tool rather than an arcade/game

Core competitive line

> **Runs builds one molecule. Circuit is the language for building molecules.**

What judges should remember

Circuit is not one trading strategy. It is a reusable way to express and execute strategies on dreamDEX Event Contracts.

4. Goals, Non-Goals, and Success Criteria

4.1 Product Goals

G1 — Make Event Contract strategies programmable

A user must be able to compose a strategy from a small set of reusable trigger/action/policy primitives.

G2 — Make strategy risk understandable before activation

The UI must show the strategy graph, maximum configured exposure, stop conditions, round limit, and action rules before the user approves anything.

G3 — Demonstrate real dreamDEX execution

The final demo must contain real testnet orders, fills/receipts, market lifecycle tracking, and settlement/redeem handling.

G4 — Demonstrate real Somnia-native automation

At least one strategy transition must be driven by a Somnia on-chain Reactivity subscription.

G5 — Use Somnia Agents without making AI the trust assumption

Natural language may generate a draft strategy, but only deterministic validated rules may execute.

G6 — Win on UX, not only architecture

A judge should understand Circuit within 10 seconds of seeing the builder screen.

4.2 Non-Goals for Hackathon v1

Circuit v1 is **not**:

- a generalized DeFi automation language
- a cross-chain product
- a pooled investment vault
- an AI alpha-generation system
- a social prediction network
- a full backtesting platform
- a copy-trading marketplace
- a high-frequency market maker
- an unbounded arbitrary smart-contract workflow engine
- a mainnet financial product

4.3 Hackathon Success Metrics

The MVP is considered successful only if all of the following are true:

1. A user creates a strategy from the visual builder.
2. A user can optionally generate the same class of strategy from natural language through Somnia Agents.
3. The UI displays deterministic rules and risk bounds before activation.
4. The user activates the strategy on Somnia Shannon Testnet.
5. Circuit has a real on-chain Reactivity subscription.
6. At least one Reactivity callback causes a verifiable state transition in Circuit.
7. Circuit causes a real dreamDEX Event Contract order to execute on testnet.

8. Circuit records the actual fill/result rather than assuming requested size equals filled size.
9. A resolved or voided market is handled correctly.
10. The strategy proceeds to a successor window or terminates according to policy.
11. Emergency pause works.
12. The demo exposes transaction links and a readable activity log.
13. The complete story can be demonstrated in 2–3 minutes.

5. Judging Strategy

The hackathon judging weights are treated as product requirements.

Criterion	Weight	Circuit target	How Circuit earns it
Innovation & Originality	20%	19/20	Strategy language/protocol instead of one hard-coded bot
Technical Implementation	25%	24/25	dreamDEX execution + on-chain Reactivity + bounded policy engine + Agent compiler
UX & Design	20%	19/20	Visual graph, risk preview, live state, human-readable activity
Business & Ecosystem Impact	20%	19/20	Makes new strategy products possible and can generate repeat Event Contract activity
Presentation & Demo	15%	14/15	One clear input → graph → activation → trigger → trade → resolve → next-window story
Target	**100%**	**95/100**	Execution quality determines final score

6. Target Users

6.1 Primary Persona — Strategy-Curious Trader

A user who understands UP/DOWN markets but does not want to maintain a bot.

Needs:

- simple strategy creation
- explicit risk limits
- non-custodial permissions
- live status
- easy pause/revoke

6.2 Secondary Persona — Strategy Builder

A technical or semi-technical user who wants reusable Event Contract workflows.

Needs:

- precise strategy semantics
- deterministic execution
- inspectable state
- shareable templates

6.3 Ecosystem Persona — dreamDEX/Somnia Builder

A developer who could later use Circuit as an execution primitive rather than reimplement strategy state machines.

Needs:

- canonical strategy format
- reusable contracts/interfaces
- clear events and state
- composability

7. Product Principles

1. **Rules before AI.** AI may interpret; deterministic rules execute.
2. **Bounded by default.** Every active strategy must have hard limits.
3. **No hidden custody.** User funds remain user-owned; Circuit receives narrowly scoped trading authority.
4. **On-chain truth before indexer truth.** Every write is gated by live market state.
5. **Actual fills before requested fills.** Strategy accounting uses confirmed execution data.
6. **Idempotent automation.** Duplicate callbacks or retries must not duplicate a strategy action.
7. **Fail closed.** Ambiguous state means skip/pause, not “best effort” trading.
8. **Explain every action.** The UI must answer “why did Circuit do this?”
9. **One great loop beats ten features.** The demo strategy must work end-to-end across rolling windows.

8. MVP Scope

8.1 P0 — Must Ship

Product

- Wallet connection on Somnia Shannon Testnet
- Live dreamDEX Event Contract discovery
- Primary demo support: BTC 15m
- Secondary support if stable: ETH 15m
- Visual strategy builder
- Strategy review/risk summary
- Activate strategy
- Pause strategy
- Resume strategy
- Stop strategy permanently
- Live strategy status page
- Human-readable activity log
- Transaction/explorer links
- One-click permission revocation guidance/status

Strategy language

P0 strategy primitives:

Triggers

- `MARKET_TRADING`
- `LAST_FILL_PRICE_ABOVE`
- `LAST_FILL_PRICE_BELOW`
- `ON_RESOLVED`
- `ON_VOIDED`

Actions

- `BUY_UP`
- `BUY_DOWN`
- `REDEEM`
- `ROLL_PERCENT`
- `NEXT_WINDOW`
- `STOP`

Policies

- `MAX_ORDER_COLLATERAL`
- `MAX_TOTAL_CAPITAL_AT_RISK`
- `MAX_ROUNDS`
- `STOP_AFTER_CONSECUTIVE_LOSSES`
- `MIN_SECONDS_TO_EXPIRY`
- `MAX_SLIPPAGE`

Integrations

- dreamDEX `@somnia-chain/markets-sdk` v0.25.0 or newer for discovery, reads, UI data, and integration verification
- Direct or adapter-based dreamDEX execution using deployed pool/module interfaces
- Somnia on-chain Reactivity subscription
- Somnia Agent natural-language-to-manifest flow

Demo template

Contrarian Roller

- Series: BTC 15m
- Trigger: last fill UP probability > 0.70

- Action: BUY DOWN
- Per-round max: 10 test collateral units
- On win: roll 50% of realized winnings/proceeds allocation into the next window
- On loss: increment consecutive-loss count
- Stop: after 2 consecutive losses
- Hard cap: 20 test collateral units total capital-at-risk
- Min time-to-expiry: 120 seconds
- Order behavior: IOC / bounded slippage

8.2 P1 — Ship Only After P0 Is Stable

- Natural-language example suggestions
- Strategy templates gallery
- Shareable read-only strategy page
- Clone strategy
- Simple realized PnL chart
- ETH 1h / BTC 1h support
- Strategy export/import JSON

8.3 P2 — Post-Hackathon

- Permissionless shared strategy registry
- Strategy creator attribution / builder fee model
- Multi-asset graphs
- AND/OR composite conditions
- Portfolio-level policies
- Backtesting and paper mode
- Strategy SDK
- Strategy vaults built on top of Circuit
- Advanced node editor
- Multi-executor decentralization
- Production subscription aggregation

9. Core User Experience

9.1 Journey A — Visual Builder

1. User connects wallet.
2. User selects `BTC · 15m`.
3. User selects or creates a trigger: `UP last fill > 70%`.
4. User selects action: `BUY DOWN`.
5. User sets max per-order collateral: `10`.
6. User adds resolution branch:
   - WIN → `ROLL 50%` → `NEXT WINDOW`
   - LOSS → increment losses → `NEXT WINDOW`
7. User sets `STOP AFTER 2 LOSSES`.
8. User sets hard cap `MAX CAPITAL AT RISK = 20`.
9. Circuit validates the graph.
10. Circuit shows risk summary.
11. User activates.
12. UI walks user through required permissions.
13. Strategy becomes `ARMED`.

9.2 Journey B — Natural Language

Input:

> “For BTC 15-minute markets, if UP trades above 70%, buy DOWN with 10. Roll half after a win and stop after two losses. Never risk more than 20.”

Flow:

1. User submits text.
2. Somnia Agent returns a canonical strategy manifest proposal.
3. Circuit validates schema and supported primitives.
4. Circuit renders the visual graph.
5. The user edits if needed.
6. The user explicitly approves.
7. Only the approved deterministic manifest is stored/activated.

UX rule

The Agent output must always be labeled:

“AI-generated draft — review before activation.”

9.3 Journey C — Live Strategy

The active strategy screen must show:

- Strategy name
- Current series/window
- State: `ARMED`, `TRIGGERED`, `ORDER_SUBMITTED`, `FILLED`, `WAITING_RESOLUTION`, `ROLLING`, `STOPPED`
- Current UP probability / relevant observed price
- Trigger rule
- Current round / max rounds
- Consecutive losses / stop threshold
- Capital at risk / hard cap
- Last action
- Next expected action
- Emergency pause button
- Explorer links

9.4 Journey D — Resolution and Next Window

When the market resolves:

1. Reactivity callback reaches Circuit.
2. Circuit verifies the market state on-chain.
3. Circuit determines resolved/voided state.
4. Circuit updates strategy result.
5. Redeem path is enabled/executed.
6. Rollover budget is computed from actual strategy accounting.
7. Stop policies are evaluated.
8. If still active, Circuit binds to the next valid window.
9. Strategy returns to `ARMED`.

10. UX / Screen Requirements

10.1 Landing

Hero:

Build strategies, not bots.

Subtext:

> Program bounded trading strategies across dreamDEX Event Contracts. React to live markets, roll across windows, and stop exactly when your rules say so.

Primary CTA: `Build a Strategy`

Secondary: `View Live Demo`

Do not lead with “AI.”

10.2 Builder Screen

Desktop-first for hackathon demo.

Layout:

- Left: node palette
- Center: strategy graph
- Right: configuration + risk panel
- Top: series selector and strategy name
- Bottom/right: `Validate` then `Review & Activate`

Minimum visible graph:

```
BTC 15m
   ↓
UP LAST FILL > 0.70?
   ↓ YES
BUY DOWN · max 10
   ↓
WAIT FOR RESOLUTION
  ↙️                ↘️
WIN                LOSS
 ↓                  ↓
ROLL 50%          LOSSES + 1
  ↘️                ↙️
   NEXT WINDOW
       ↓
STOP IF LOSSES = 2
```

10.3 Review & Activate

This is a critical trust screen.

Must display:

- “What can Circuit do?”
- “What can Circuit not do?”
- Maximum per-order spend
- Maximum total capital at risk
- Maximum number of rounds
- Stop-after-losses rule
- Series/cadence
- Slippage limit
- Minimum time-to-expiry
- Permission status
- Reactivity subscription status

Activation CTA remains disabled until all required checks pass.

10.4 Live Activity Timeline

Example:

```
18:03:04  Strategy armed for BTC 15m
18:04:11  Observed UP fill at 0.712
18:04:11  Trigger matched: > 0.700
18:04:12  BUY DOWN submitted · max 10.00
18:04:13  Filled 12.8 DOWN · 9.74 collateral spent
18:15:02  Market resolved: DOWN
18:15:03  Reactivity callback verified
18:15:05  Position redeemed
18:15:06  Roll budget computed: 5.13
18:15:07  Next BTC 15m window armed
```

Every automated line should be derived from a transaction, event, or deterministic state transition.

10.5 Error UI

Errors must be translated into clear user states:

- `No liquidity — waiting`
- `Market locked before execution — skipped`
- `Price moved beyond slippage — not executed`
- `Risk limit reached — strategy stopped`
- `Reactivity subscription underfunded — automation paused`
- `Permission revoked — strategy paused`
- `Market voided — neutral rollover policy applied`

11. Strategy DSL

Circuit stores a canonical, deterministic manifest. Natural language and visual editing are merely two ways to produce this manifest.

11.1 Canonical Manifest v1

```
{
  "version": 1,
  "name": "Contrarian Roller",
  "series": {
    "asset": "BTC",
    "intervalSec": 900
  },
  "trigger": {
    "type": "LAST_FILL_PRICE_ABOVE",
    "value": "0.700"
  },
  "action": {
    "type": "BUY_DOWN",
    "maxCollateral": "10.0",
    "maxSlippageBps": 200
  },
  "resolution": {
    "onWin": {
      "rollPercent": 50
    },
    "onLoss": {
      "incrementConsecutiveLosses": true
    },
    "onVoid": {
      "treatAsLoss": false,
      "treatAsWin": false
    }
  },
  "policy": {
    "maxTotalCapitalAtRisk": "20.0",
    "maxRounds": 5,
    "stopAfterConsecutiveLosses": 2,
    "minSecondsToExpiry": 120
  }
}
```

11.2 Determinism Rules

- Decimal values must be parsed into integer fixed-point units before on-chain storage.
- Float arithmetic must not be used for order price/tick construction.
- Every manifest must include a hard capital bound.
- Every manifest must include a round bound or another finite termination condition.
- Unsupported fields invalidate the manifest.
- Unknown enum values invalidate the manifest.
- Agent output is never trusted without validation.
- Active manifests are immutable. Editing creates a new strategy version.

11.3 Graph Constraints

Hackathon v1 allows a structured finite-state graph, not arbitrary loops.

Permitted loop:

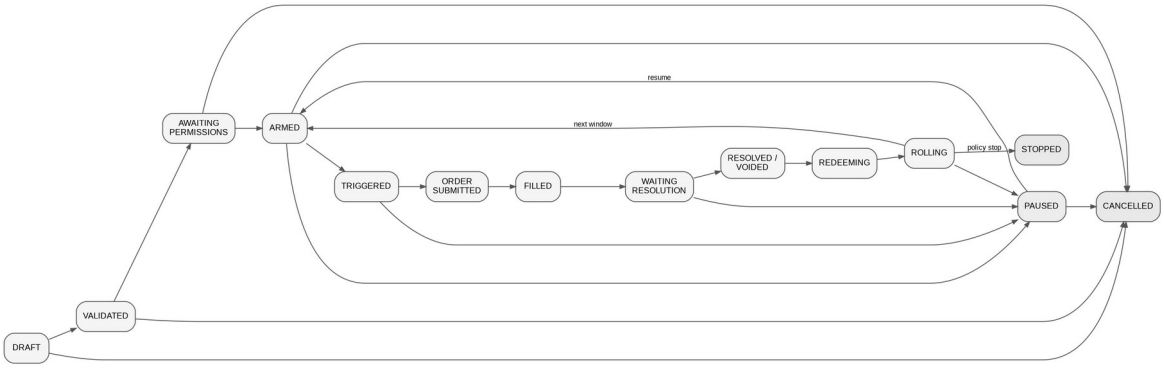
`resolution → next rolling window → same trigger/action`

The loop is bounded by:

- max rounds
- capital cap
- loss stop

No user-defined arbitrary contract call node is permitted.

12. Strategy State Machine



Canonical states:

```
DRAFT
  ↓
VALIDATED
  ↓
AWAITING_PERMISSIONS
  ↓
ARMED
  ↓ trigger matched
TRIGGERED
  ↓
ORDER_SUBMITTED
  ↓
FILLED / SKIPPED
  ↓
WAITING_RESOLUTION
  ↓
RESOLVED / VOIDED
  ↓
REDEEMING
  ↓
ROLLING
  ├── policy allows → ARMED(next window)
  └── policy stops  → STOPPED

Any active state → PAUSED
Any state except STOPPED → CANCELLED
```

Terminal states

- `STOPPED`: policy ended strategy normally
- `CANCELLED`: user permanently stopped strategy

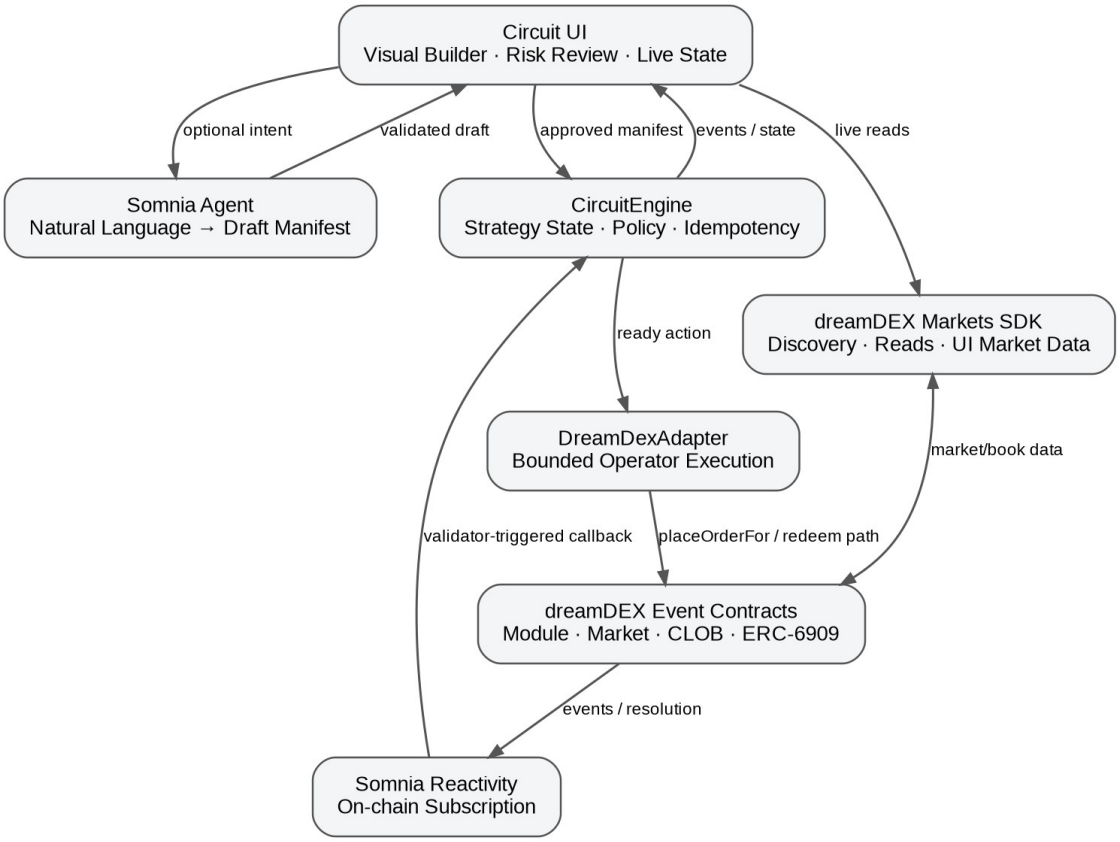
Idempotency

Each strategy action must be keyed by at least:

`strategyId + round + marketId + actionType`

A processed action key cannot execute twice.

13. Technical Architecture



13.1 High-Level Components

Frontend

Recommended stack:

- Next.js / React / TypeScript
- wagmi + viem
- a node graph library such as React Flow
- dreamDEX Markets SDK for live market discovery/read UX

Responsibilities:

- wallet connection
- builder UI
- strategy validation preview
- permissions workflow
- live market data
- strategy state display
- explorer/deep links
- Agent prompt UX

`CircuitRegistry`

Responsibilities:

- strategy IDs
- owner mapping
- immutable manifest hash/version
- strategy metadata/events

May be merged into `CircuitEngine` for hackathon simplicity.

`CircuitEngine`

The protocol core.

Responsibilities:

- deterministic strategy state
- policy enforcement
- round accounting
- event/callback idempotency
- current market binding
- trigger evaluation
- transition logic
- dreamDEX execution authorization path
- pause/stop

`DreamDexAdapter`

Responsibilities:

- translate Circuit actions (`BUY_UP`, `BUY_DOWN`) into exact dreamDEX pool/module calls
- resolve current market/pool from authoritative on-chain data
- snap price and quantity to live tick/lot rules
- gate every write on `Trading` state
- use IOC for P0 taker actions
- never hard-code per-window market/pool bindings

`CircuitReactivityHandler`

May be merged into `CircuitEngine`.

Responsibilities:

- accept only valid Somnia Reactivity invocations
- decode subscribed event payloads
- map event → strategy/round
- call idempotent transition functions
- emit transparent Circuit events

Agent Adapter

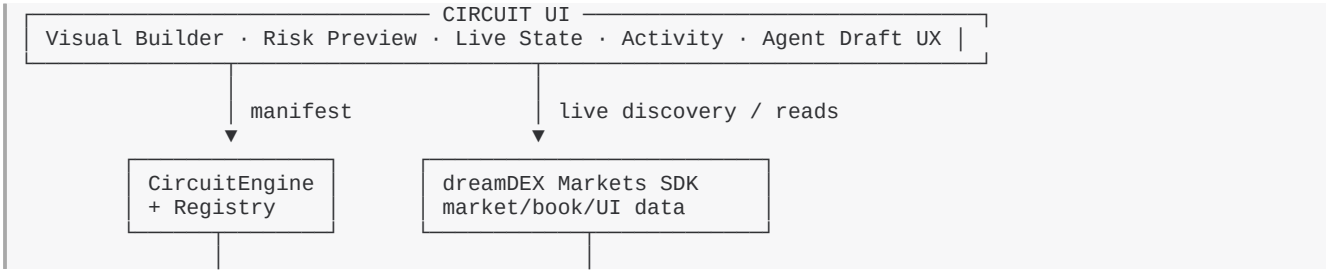
Responsibilities:

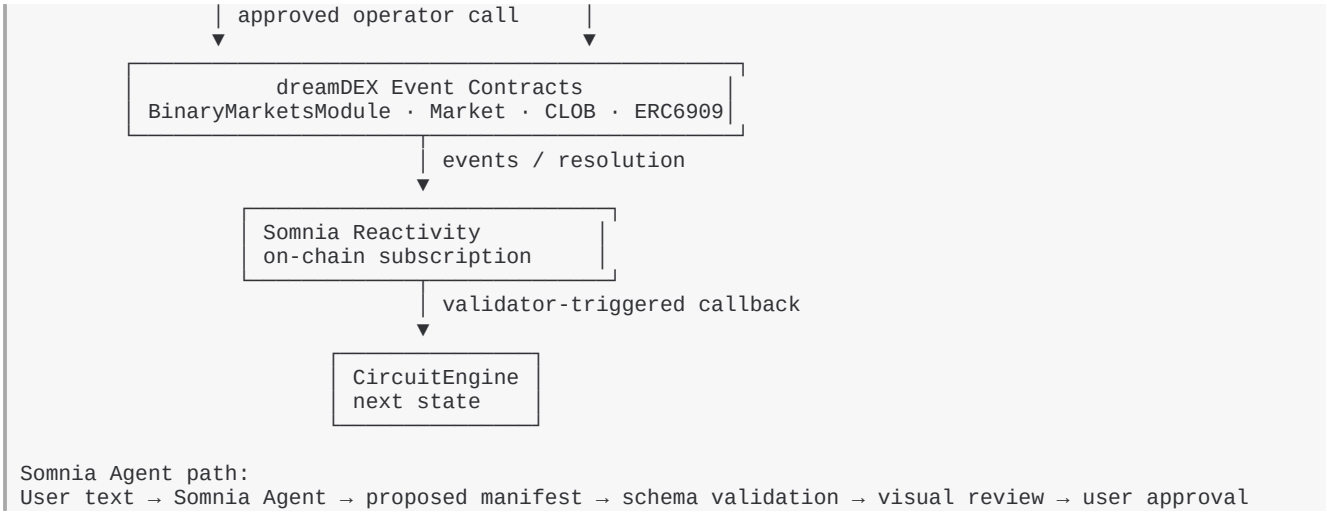
- send constrained prompt to Somnia LLM Agent
- receive deterministic response
- parse canonical manifest
- reject malformed/unsupported output
- never activate automatically

Optional UI Index Layer

A lightweight local/server index may be used only for presentation/history. It must never be the source of truth for execution authorization or market status.

13.2 Architecture Diagram






```
Somnia Agent path:
User text → Somnia Agent → proposed manifest → schema validation → visual review → user approval
```

14. Non-Custodial Execution Model

Circuit must not take unrestricted custody of user funds.

14.1 Preferred P0 Model

`CircuitEngine` is approved by the user as a dreamDEX **operator contract** for the minimum capability needed to place orders on the user's behalf.

The dreamDEX operator model is important because orders remain owned by the user and proceeds settle to the user, not to the operator.

P0 should request only the `placeOrderFor` capability unless another capability becomes technically necessary.

Current documented selector:

`placeOrderFor → 0x80054449`

Because P0 orders use IOC, Circuit should not require persistent order cancellation rights.

14.2 Collateral Approval

Under dreamDEX auto-pull, the user also needs the required ERC-20 allowance to the relevant pool for the input collateral.

Circuit UI must:

1. resolve the live pool
2. check required allowance
3. request only what is necessary for the demo/product flow
4. show the user how to revoke

Do not assume a market pool address is permanent. dreamDEX recycles pools across rolling windows; always resolve current bindings.

14.3 Policy Enforcement

A permissionless caller or Reactivity callback may ask Circuit to execute an action, but `CircuitEngine` must enforce:

- caller/callback validity where applicable
- strategy owner
- active state
- correct round
- correct market binding
- trigger satisfied
- market status = Trading before order write

- min time-to-expiry
- max order collateral
- max total capital-at-risk
- max rounds
- loss-stop state
- slippage bound
- duplicate-action prevention

The person/system submitting the transaction must not be able to widen strategy limits.

15. dreamDEX Integration Requirements

15.1 SDK

Use `@somnia-chain/markets-sdk` **v0.25.0 or newer** for application-level discovery/reads and integration testing.

15.2 Market Status

Never rely on indexer status for a write.

Immediately before every execution:

- resolve the current market
- read live on-chain market state
- require `Trading`
- verify expiry buffer

15.3 Market Identity

Key Circuit state by `marketId` and strategy series identity.

Do not key by pool address because pools may be recycled across windows.

15.4 Price Semantics

UP and DOWN share one book expressed in UP probability terms.

For P0 trigger semantics:

`LAST_FILL_PRICE_ABOVE 0.70` means the observed UP-term `OrderFilled.fillPrice` is above the configured threshold.

This avoids ambiguous “oracle price” wording.

15.5 Order Type

Use IOC for the P0 strategy.

Reasons:

- avoids stale resting orders
- no cancellation requirement for unfilled remainder
- demo behavior is easier to reason about
- slippage can be bounded explicitly

15.6 Tick and Lot Safety

All prices/quantities must be snapped to live tick/lot grids using integer math.

Never build 18-decimal order prices with JavaScript floating-point multiplication by `1e18`.

15.7 Fill Accounting

Requested quantity is not execution truth.

Circuit accounting must use actual fills / confirmed outcome balances / transaction results.

Partial IOC fill behavior:

- record actual fill
- unfilled remainder is ignored
- risk accounting uses actual committed/filled collateral according to final execution data

15.8 Settlement and Redeem

Settled markets leave the normal live market list.

Circuit must record every `marketId` it traded so the redeem path never depends on rediscovering a settled market from the live list.

Resolved:

- redeem winning outcome held

Voided:

- both outcomes may be redeemable at 0.5
- do not count a void as win or loss in P0
- continue or stop according to remaining policy/round rules

15.9 Successor Window

Every cycle must re-resolve the current live window for the selected series.

Circuit must not assume the next market by incrementing a local ID unless validated against the module/SDK.

16. Somnia Reactivity Requirements

16.1 Mandatory Integration

The final submission must include at least one **on-chain** Reactivity subscription.

An off-chain WebSocket subscription alone does not satisfy the Circuit P0 definition.

16.2 P0 Subscription Use

Preferred two-event model:

Market activity trigger

Subscribe to the relevant dreamDEX pool `OrderFilled` event.

Use the event's fill price as trigger evidence for `LAST_FILL_PRICE_ABOVE/BELOW` strategies.

Resolution progression

Subscribe to the stable resolution-related event exposed by the deployed dreamDEX market/settlement/oracle flow and call the Circuit handler.

Implementation must validate the exact event ABI during the Day-1 integration spike. Regardless of event source, the handler must re-read market state before applying resolved/voided transitions.

16.3 Handler Security

- validate invocation path/source required by Somnia Reactivity
- reject arbitrary direct callback spoofing
- validate emitter/origin/topic where applicable
- re-read current market state
- idempotently process callbacks

Somnia docs identify the Reactivity precompile as `0x0100`; implementation must verify deployed/testnet behavior against the current docs before finalizing access control.

16.4 Subscription Funding

On-chain subscriptions require funded gas parameters and a minimum balance.

UI/admin tooling must surface:

- subscription ID
- active/inactive
- balance/funding state if available
- handler address

A low-gas or underfunded subscription must produce a visible `AUTOMATION_PAUSED` state rather than silent failure.

16.5 Backstop

Circuit must expose a permissionless `sync`/`poke` path that can advance a strategy from verifiable on-chain state if a Reactivity callback is missed.

This is a liveness backstop only. It must not create a second source of truth.

17. Somnia Agent Requirements

17.1 Purpose

The Agent is a strategy compiler UX feature.

Input:

plain-English strategy intent

Output:

one supported Circuit manifest only

17.2 Allowed Agent Scope

The Agent may:

- identify series (`BTC 15m`, etc.)
- identify supported threshold trigger
- map “buy up/down” language to action enum
- extract explicit budget/stop limits
- format a manifest

The Agent may not:

- infer missing capital limits silently
- choose an asset the user did not request
- invent a risk appetite
- activate the strategy
- alter a live strategy

17.3 Missing Required Limits

If the prompt omits a mandatory safety field, the Agent/compiler must return an incomplete draft and UI asks the user to provide the missing value.

Example:

User: “Buy DOWN whenever UP is above 70%.”

UI response:

> “Strategy understood. Add a max order amount and hard capital-at-risk cap before activation.”

17.4 Validation

Agent output must pass:

1. JSON parse
2. version check
3. enum validation
4. numeric bound validation
5. finite termination validation
6. supported series check
7. deterministic canonicalization

Then the UI renders the graph for human approval.

18. Smart Contract Interface — Proposed

Names may change in code, semantics may not.

18.1 Strategy Struct (conceptual)

```
struct Strategy {
    address owner;
    uint8 status;
    bytes32 manifestHash;
    bytes32 currentMarketId;
    uint32 intervalSec;
    uint8 assetId;
    uint8 triggerType;
    uint256 triggerValue;
    uint8 actionType;
    uint256 maxOrderCollateral;
    uint16 maxSlippageBps;
    uint256 maxTotalCapitalAtRisk;
    uint16 maxRounds;
    uint16 round;
    uint16 consecutiveLosses;
    uint16 stopAfterLosses;
    uint32 minSecondsToExpiry;
    uint16 rollPercentBps;
    uint256 cumulativeCapitalUsed;
    uint256 currentPositionSize;
}
```

18.2 Required Functions (conceptual)

```
createStrategy(...)
activateStrategy(strategyId)
pauseStrategy(strategyId)
resumeStrategy(strategyId)
cancelStrategy(strategyId)

handleMarketFill(...)
handleResolution(...)

executeReadyAction(strategyId, ...)
syncStrategy(strategyId, marketId)

bindNextWindow(strategyId, marketId)
```

Exact ABI should be optimized during implementation.

18.3 Required Events

```
StrategyCreated
StrategyActivated
StrategyPaused
StrategyResumed
StrategyStopped
StrategyCancelled
MarketBound
TriggerMatched
OrderRequested
OrderExecuted
OrderSkipped
RoundResolved
```

```
PositionRedeemed
RolloverComputed
RiskLimitReached
ReactivityCallbackProcessed
```

These events power the UI activity timeline.

19. Risk and Security Requirements

19.1 Mandatory Hard Limits

Every strategy requires:

- per-order cap
- total capital-at-risk cap
- finite round count
- min expiry buffer
- slippage cap

Loss stop is mandatory for the main demo template.

19.2 No Arbitrary Calls

User strategy manifests cannot encode arbitrary target addresses, calldata, delegatecalls, external contracts, token transfers, or approvals.

19.3 Permission Scope

Prefer operator permission for `placeOrderFor` only.

Do not request `cancelOrderFor` or `reduceOrderFor` unless P0 implementation proves it is needed.

19.4 User Escape Hatch

The user can always:

- pause in Circuit
- cancel strategy
- revoke Circuit operator approval in dreamDEX registry
- revoke collateral approval to pool

The UI should surface these controls visibly.

19.5 Reentrancy / External Calls

Execution paths that call dreamDEX must use reentrancy protection and update internal idempotency markers before/around external interactions according to checks-effects-interactions discipline.

19.6 Duplicate Events

Duplicate Reactivity notifications or transaction retries cannot create duplicate orders.

19.7 Agent Trust Boundary

Agent output is untrusted input until canonical validation and user approval.

20. Failure Modes and Required Behavior

Failure	Required behavior
No resting liquidity	Remain armed or skip round; never exceed slippage
Partial IOC fill	Record actual fill only
Market locks before tx	Skip safely; no retry on locked market

Failure	Required behavior
Indexer says Trading but chain says Locked	Chain wins; do not trade
Pool recycled	Re-resolve by marketId/series before writes
Strategy trigger fires twice	Idempotency prevents second action
Reactivity callback duplicated	No duplicate transition
Reactivity underfunded	Visible paused/degraded state + permissionless sync backstop
Agent returns malformed JSON	Reject; no activation
Agent omits risk cap	Require user input
Market voided	Redeem correctly, no win/loss increment
User revokes operator permission	Pause and show permission error
User revokes token allowance	Pause and show funding permission error
Slippage exceeds configured max	Do not execute
Risk cap reached	Stop immediately
Max rounds reached	Stop normally
Consecutive loss threshold reached	Stop normally
Redeem transaction fails	Keep claimable state and retry safely

21. Data Model / Frontend Types

```
type StrategyStatus =
  | 'DRAFT'
  | 'VALIDATED'
  | 'AWAITING_PERMISSIONS'
  | 'ARMED'
  | 'TRIGGERED'
  | 'ORDER_SUBMITTED'
  | 'FILLED'
  | 'WAITING_RESOLUTION'
  | 'REDEEMING'
  | 'ROLLING'
  | 'PAUSED'
  | 'STOPPED'
  | 'CANCELLED'

type TriggerType =
  | 'MARKET_TRADING'
  | 'LAST_FILL_PRICE_ABOVE'
  | 'LAST_FILL_PRICE_BELOW'
  | 'ON_RESOLVED'
  | 'ON_VOIDED'

type ActionType =
  | 'BUY_UP'
  | 'BUY_DOWN'
  | 'REDEEM'
  | 'ROLL_PERCENT'
  | 'NEXT_WINDOW'
  | 'STOP'
```

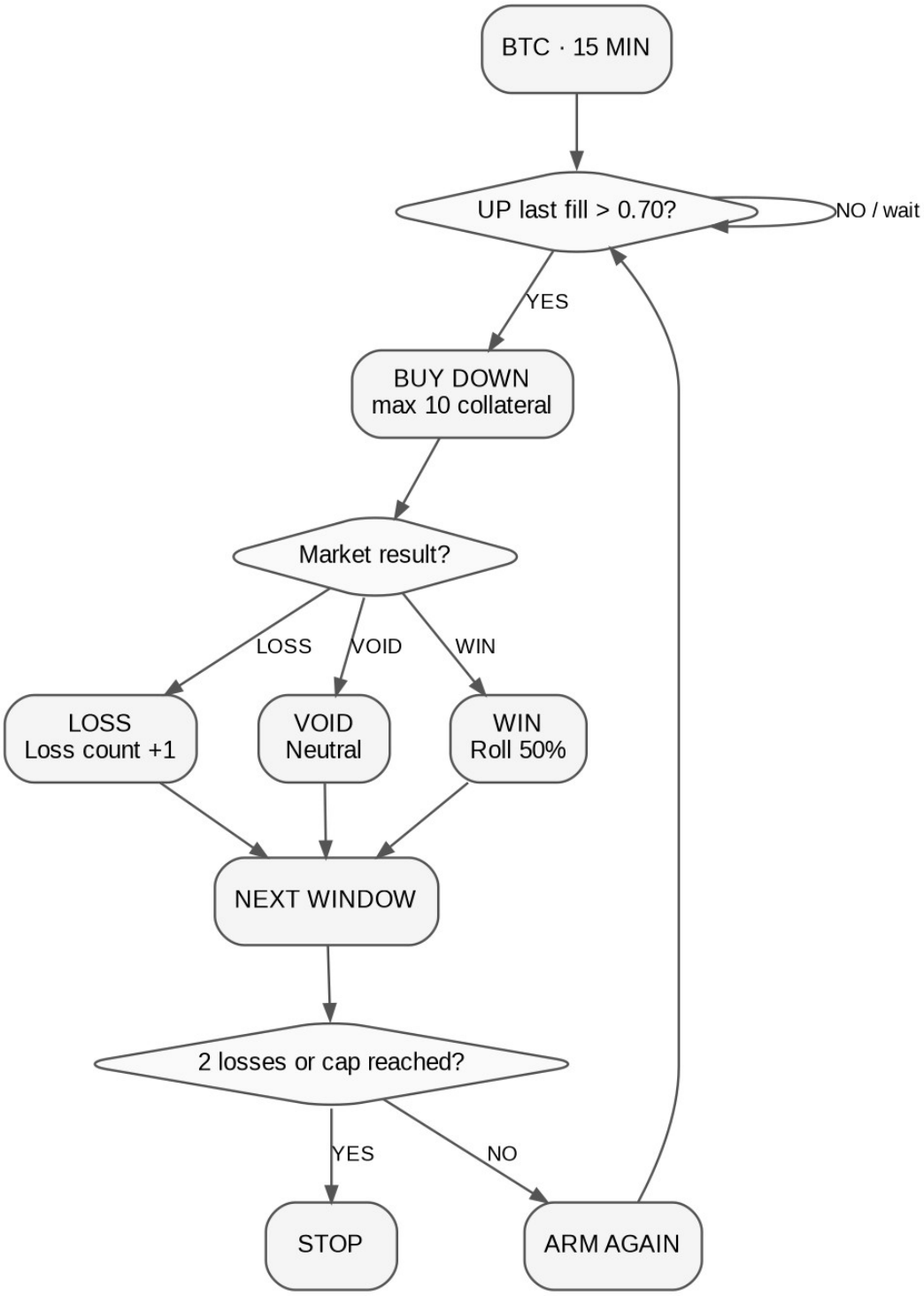
Activity item:

```
interface ActivityItem {
  id: string
  strategyId: string
  round: number
  marketId?: `0x${string}`
  type: string
  summary: string
```

```
  txHash?: `0x${string}`
  timestamp: number
  source: 'CONTRACT' | 'REACTIVITY' | 'DREAMDEX' | 'AGENT' | 'UI'
}
```

22. Example Strategies

22.1 Contrarian Roller — P0 Demo



```
IF BTC 15m UP last fill > 0.70
→ BUY DOWN, max 10
→ wait for resolution
→ WIN: roll 50%
→ LOSS: loss counter +1
→ next window
→ stop at 2 consecutive losses or 20 total capital-at-risk
```

22.2 Momentum Follow — Template

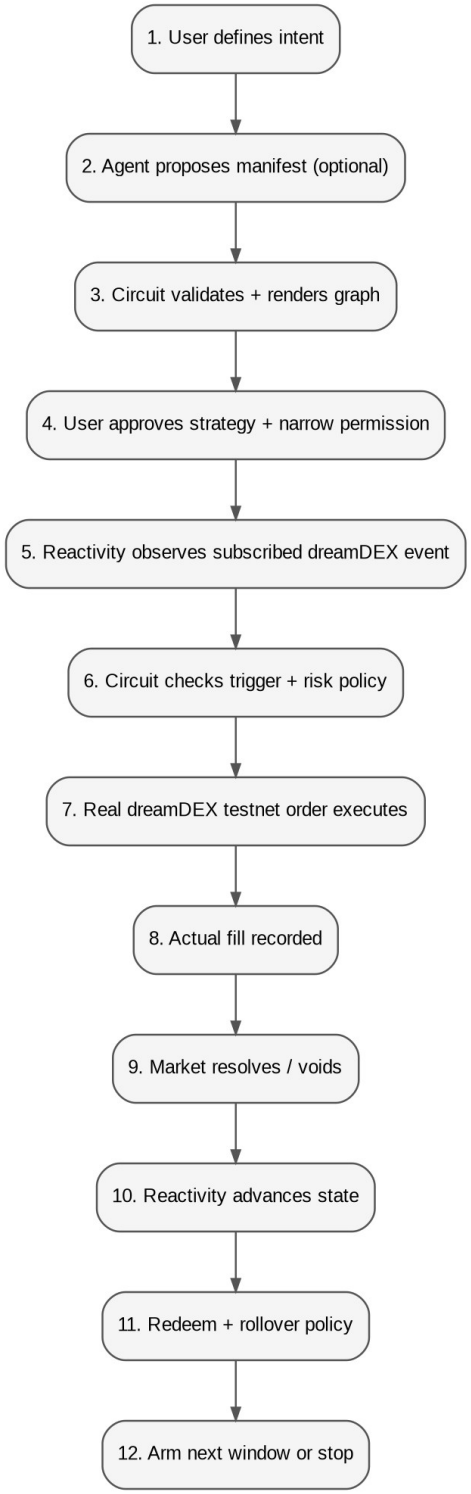
```
IF BTC 15m UP last fill > 0.60
→ BUY UP, max 5
→ next window after resolution
→ stop after 3 rounds
```

22.3 Conditional Ladder — Template

```
Round 1: IF UP < 0.40 → BUY UP 4
Round result WIN → next window budget 6
Round result LOSS → stop
Hard cap 10
```

For hackathon implementation, templates must compile to the same v1 primitive set. Do not add special-case code per template.

23. Demo Script — 2 to 3 Minutes



0:00–0:15 — Problem

> “Every dreamDEX Event Contract gives you one decision: Up or Down. Today, every multi-step strategy becomes another custom bot. Circuit makes those strategies programmable.”

Show builder immediately.

0:15–0:35 — Natural Language → Graph

Type:

> "If BTC 15m UP trades above 70%, buy DOWN with 10. Roll half after a win and stop after two losses. Never risk more than 20."

Show:

Somnia Agent → proposed manifest → graph.

Say:

> "The Agent only interprets intent. It never controls funds."

0:35–0:55 — Risk Review

Highlight:

- max order: 10
- hard cap: 20
- stop after 2 losses
- max rounds
- min expiry buffer
- max slippage

Activate.

0:55–1:20 — Real Trigger and dreamDEX Trade

Show live market.

`OrderFilled` crosses 0.70.

Show Reactivity callback / Circuit activity:

`Trigger matched → BUY DOWN`.

Show real testnet transaction and fill.

1:20–1:50 — Resolution

Use a prepared short-window demo state if necessary.

Market resolves.

Show:

- on-chain resolution
- Reactivity callback
- Circuit updates result
- redeem
- rollover budget

1:50–2:10 — Next Window

Show successor BTC 15m market bound and strategy returning to `ARMED`.

2:10–2:30 — Close

Show graph + live state + activity.

Final line:

> **"dreamDEX built Event Contracts. Circuit makes them programmable."**

24. Acceptance Tests

24.1 Strategy Validation

- ☐ Reject missing hard capital cap
- ☐ Reject zero max rounds
- ☐ Reject unsupported action
- ☐ Reject trigger probability outside 0–1
- ☐ Reject roll percent > 100%
- ☐ Reject slippage above product-defined max
- ☐ Canonical manifest hash is stable

24.2 dreamDEX

- ☐ Discover current BTC 15m market
- ☐ Read on-chain status before write
- ☐ Fetch current pool from authoritative source
- ☐ Snap tick/lot correctly
- ☐ Execute BUY UP test order
- ☐ Execute BUY DOWN test order
- ☐ Handle IOC partial/no fill
- ☐ Record tx receipt
- ☐ Record actual position/fill
- ☐ Detect settled traded market
- ☐ Redeem resolved winner
- ☐ Handle void path
- ☐ Find successor window

24.3 Reactivity

- ☐ Create real on-chain subscription
- ☐ Callback reaches Circuit handler
- ☐ Direct spoofed callback rejected
- ☐ Duplicate callback idempotent
- ☐ Underfunded subscription state detected/surfaced
- ☐ Permissionless sync backstop works

24.4 Operator / Permissions

- ☐ User can approve Circuit operator capability
- ☐ Circuit can place order for user only within active strategy policy
- ☐ User funds/proceeds remain owned by user
- ☐ Revoking operator permission blocks execution
- ☐ UI detects revoked permission

24.5 Agent

- ☐ Valid English prompt compiles to valid manifest
- ☐ Missing risk limits produce incomplete draft, not invented values
- ☐ Malformed response rejected
- ☐ User must explicitly approve final graph

24.6 UX

- ☐ Judge can understand active strategy in <10 seconds
- ☐ Risk bounds visible without scrolling on review screen
- ☐ Emergency pause visible on active screen

☐ Every trade has explorer link
☐ Activity log explains why action occurred

25. Implementation Plan — 4 September to 8 September 2026

Day 1 — Sep 4: Integration Spike + Skeleton

Must finish:

- repository structure
- current official SDK setup (`>=0.25.0`)
- live market discovery
- BTC 15m read path
- exact BUY_UP and BUY_DOWN testnet transaction path
- operator permission spike
- identify exact Reactivity subscription emitter/topic for P0
- deploy minimal handler and receive one callback
- lock final contract interface

Do not spend the day on visual polish.

Day 2 — Sep 5: Circuit Engine

- manifest validation
- strategy storage
- state machine
- risk policy checks
- idempotency
- dreamDEX adapter
- operator-based execution
- IOC/price/lot handling
- manual end-to-end strategy action
- pause/stop

Definition of done: a manually triggered valid strategy causes a real bounded dreamDEX testnet trade.

Day 3 — Sep 6: Reactivity + Agent + Builder

- on-chain subscription wired to engine
- resolution progression
- successor-window state
- Agent compiler path
- visual builder
- risk review screen
- activity timeline

Definition of done: input → graph → activate → reactive transition → real trade.

Day 4 — Sep 7: Full Loop + Polish

- complete settlement/redeem/roll loop
- full Contrarian Roller demo
- error states
- explorer links
- UI polish
- architecture diagram
- README
- test suite
- rehearse demo repeatedly

Submission Day — Sep 8

- final testnet deployment

- record 2–3 minute demo
- edit only for pacing/clarity
- verify public GitHub
- add contract addresses and tx proofs
- submit DoraHacks well before 21:00 deadline
- buffer for broken RPC / video upload / form issues

26. Repository Structure

Recommended:

```
circuit/
├── apps/
│   └── web/
│       ├── app/
│       ├── components/
│       │   ├── builder/
│       │   ├── strategy/
│       │   └── activity/
│       └── lib/
│           ├── dreamdex/
│           ├── agent/
│           └── contracts/
├── contracts/
│   ├── src/
│   │   ├── CircuitEngine.sol
│   │   ├── CircuitReactivityHandler.sol
│   │   ├── DreamDexAdapter.sol
│   │   └── interfaces/
│   ├── script/
│   └── test/
├── packages/
│   ├── strategy-dsl/
│   └── shared/
├── docs/
│   ├── ARCHITECTURE.md
│   ├── STRATEGY_DSL.md
│   └── DEMO.md
├── README.md
└── LICENSE
```

If monorepo setup becomes a time sink, simplify immediately. Working integration is more important than repository aesthetics.

27. README Requirements

README must communicate the product in this order:

1. One sentence
2. Why Circuit exists
3. 30-second demo GIF/video
4. Architecture diagram
5. How strategy execution works
6. dreamDEX integration proof
7. Somnia Reactivity integration proof
8. Somnia Agent integration
9. Security / non-custodial model
10. Contract addresses
11. Example transactions
12. Local setup
13. Strategy DSL
14. Hackathon scope vs future roadmap

First paragraph:

> **Circuit is a programmable strategy layer for dreamDEX Event Contracts. Users compose bounded rules such as “if UP trades above 70%, buy DOWN, roll half after a win, stop after two losses,” and Circuit executes the approved strategy across rolling markets using dreamDEX for execution and Somnia Reactivity for event-driven progression.**

28. Submission Copy

Project Name

Circuit

One-Liner

Circuit turns dreamDEX Event Contracts into programmable, bounded strategies that can react to live markets and continue automatically across rolling windows.

Short Description

Circuit is a programmable strategy layer for dreamDEX Event Contracts. Instead of hard-coding one trading bot, users compose deterministic rules such as probability triggers, UP/DOWN actions, rollover behavior, loss stops, round limits, and capital caps. Somnia Reactivity drives on-chain event-based strategy transitions, while Somnia Agents can convert natural-language intent into a strategy draft that the user must review before activation. Execution remains bounded by user-approved rules and uses real dreamDEX Event Contracts on Somnia testnet.

Key Differentiator

Other projects build individual strategies on dreamDEX. Circuit makes strategies themselves programmable.

29. Future Vision

Circuit can become a reusable strategy protocol beneath multiple Event Contract products.

Future builders could express:

- rolling directional strategies
- conditional sequences
- risk-managed hedges
- vault mandates
- agent-created but policy-bounded workflows
- strategy templates

Circuit should not attempt to own all of these products. It should become the **execution language and policy layer** they can use.

Long-term product direction:

30. Hard Product Decisions — Final

These decisions are locked for the hackathon:

1. **Name:** Circuit
2. **Category:** Programmable Event Contract strategy protocol/tool
3. **Core venue:** dreamDEX Event Contracts

4. **Core Somnia technology:** on-chain Reactivity
5. **Agent role:** small natural-language compiler, not trading oracle
6. **Primary demo series:** BTC 15m
7. **Primary demo strategy:** Contrarian Roller
8. **Execution:** real testnet trades
9. **Risk model:** bounded and user-visible
10. **Custody model:** no unrestricted Circuit custody; use narrowly scoped dreamDEX operator mechanics
11. **Order style:** IOC for P0
12. **Strategy graph:** finite, constrained v1 DSL
13. **Main UI:** visual strategy builder + live strategy state
14. **Main pitch:** “Other projects build strategies. Circuit makes strategies programmable.”
15. **Main close:** “dreamDEX built Event Contracts. Circuit makes them programmable.”

Appendix A — Current Technical Facts to Respect

- dreamDEX Event Contracts are integrated through `@somnia-chain/markets-sdk`; current docs require v0.25.0 or newer.
- Event Contract markets have rolling windows and must be keyed by `marketId`/series, not cached pool addresses.
- Current Event Contract lifecycle includes Listed, Trading, Locked, Resolved, and Voided.
- Current docs list BTC and ETH with 15-minute and 1-hour rolling windows.
- Event Contract order books use UP probability terms; DOWN is the inverse side of the same book.
- `OrderFilled` includes `fillPrice`, which is suitable trigger evidence for the P0 threshold primitive.
- Settled markets are not discovered through the normal live list; Circuit must remember markets it traded for redeem.
- On-chain Reactivity subscriptions invoke Solidity handlers and require funding/gas configuration.
- Reactivity is currently documented as testnet-only.
- dreamDEX operator permissions support contract/key-based `placeOrderFor` authority while orders/funds remain owned by the user.
- Somnia Agents support deterministic validator-executed inference with asynchronous callbacks; Circuit uses this only for manifest drafting.

Appendix B — Official References

Technical implementation must verify against current docs immediately before deployment because both dreamDEX and Somnia documentation are living systems.

1. dreamDEX — Building on Event Contracts
https://app.dreamdex.io/docs/developers/event-contracts
2. dreamDEX — Event Contract Recipes
https://app.dreamdex.io/docs/developers/event-contracts/recipes
3. dreamDEX — Market Structure & Lifecycle
https://app.dreamdex.io/docs/developers/event-contracts/market-structure
4. dreamDEX — Contracts & Addresses
https://app.dreamdex.io/docs/developers/event-contracts/contracts-and-addresses
5. dreamDEX — Operators & Session Keys
https://app.dreamdex.io/docs/trading/spot/operators
6. dreamDEX — Contract Events
https://app.dreamdex.io/docs/developers/contracts/events
7. Somnia — Reactivity
https://docs.somnia.network/developer/reactivity
8. Somnia — Reactivity Subscription Management
https://docs.somnia.network/developer/reactivity/tooling/subscription-management
9. Somnia — On-chain Reactivity Tutorial

https://docs.somnia.network/developer/reactivity/tutorials/solidity-on-chain-reactivity-tutorial

10. Somnia — Agents
https://docs.somnia.network/agents

Appendix C — Current Deployment Notes (Verify Before Use)

From current dreamDEX documentation at time of PRD lock:

- Somnia Shannon Testnet chain ID: `50312`
- BinaryMarketsModule: `0x3ecC694Cef705358864a646142ac17A90E29e388`
- MarketsCore: `0x2802504314685D89bF6C992CA5a8e7cC78bc0294`
- BinarySettlement: `0xbF4a49e0Dfd092e5FBE8E5761064C49533e6Ed23`
- OutcomeToken6909: `0xB52c5934113Af5c0Bb20eb3C72290C8215f755b9`
- OracleHub: `0xe40db387cC98601Dd11bd634fF2f3AD5686dE32b`
- CollateralRouter: `0xbC0C9834B15ACE38bB50dDaa7d7f7C7CC4DC183C`
- Testnet OperatorPermissionsRegistry: `0x15C7e8CE38F021c5b45d098AaD788f63090bF20A`
- Reactivity precompile documented address: `0x0100`

Do not hard-code per-window market/pool addresses.

Appendix D — Definition of “Done”

Circuit is done for this hackathon when a judge can watch the following without explanation gaps:

```
USER INTENT
   ↓
VALID STRATEGY GRAPH
   ↓
VISIBLE RISK BOUNDS
   ↓
USER ACTIVATION + NARROW PERMISSION
   ↓
REAL DREAMDEX MARKET
   ↓
ON-CHAIN REACTIVITY EVENT
   ↓
CIRCUIT STATE TRANSITION
   ↓
REAL DREAMDEX TESTNET ORDER
   ↓
ACTUAL FILL RECORDED
   ↓
MARKET RESOLUTION
   ↓
REDEEM / RESULT
   ↓
POLICY CHECK
   ↓
NEXT WINDOW OR STOP
```

If that exact chain works reliably and the UI communicates it clearly, do not add more scope.